

8086 – Como ler uma String

Para ler uma string no microprocessador **8086**, a forma mais comum e padrão é utilizando o **Assembly x86** em ambiente DOS através da **Interrupção 21h** (gerenciada pelo sistema operacional).

A função específica para isso é a **Função 0Ah** (Leitura Bufferizada do Teclado). Ela exige que você defina um espaço na memória (um buffer) com um formato muito específico antes de fazer a chamada.

O Formato do Buffer (O Segredo da Função 0Ah)

Você não pode simplesmente apontar para um espaço vazio. O 8086 espera que o seu buffer tenha 3 seções obrigatórias:

1. **Byte 1 (Tamanho Máximo):** Você define quantos caracteres o buffer pode aceitar (incluindo o *Enter*).
2. **Byte 2 (Tamanho Real):** Fica vazio (?). O próprio DOS preencherá este byte com o número de caracteres que o usuário realmente digitou.
3. **Byte 3 em diante (Os Caracteres):** Onde o texto digitado será guardado.

Código Prático em Assembly 8086

Abaixo está o código estruturado para criar o buffer, ler a string do teclado e, em seguida, exibi-la de volta na tela:

```
.model small
.stack 100h

.data
; 1. DEFINIÇÃO DO BUFFER
; [Tamanho Máximo: 20] , [Tamanho Real: ?] , [Espaço para 20 caracteres
preenchidos com '$']
buffer      db 20, 0, 20 dup('$')

pula_linha  db 10, 13, '$' ; Caracteres para quebrar linha (LF, CR)

.code
main proc
; Inicializa o segmento de dados (obrigatório)
mov ax, @data
mov ds, ax

; 2. CONFIGURAÇÃO DA LEITURA (Função 0Ah)
mov ah, 0Ah      ; Código da função de leitura de string
lea dx, buffer   ; DX deve apontar para o início do buffer
int 21h          ; Chama a interrupção do DOS (pausa e espera o usuário
digitar)

; 3. QUEBRA DE LINHA (Apenas estética)
mov ah, 09h
lea dx, pula_linha
int 21h

; 4. EXIBIÇÃO DA STRING DIGITADA
; Como o buffer começa com o tamanho máximo e tamanho real, o texto de fato
```

```

; começa 2 bytes depois. Por isso, avançamos o ponteiro DX em 2 posições.
mov ah, 09h          ; Função 09h exibe string na tela (exige terminar com
'$')
lea dx, buffer
add dx, 2            ; Avança 2 bytes para pular os metadados do buffer
int 21h              ; Exibe o que foi digitado

; Finaliza o programa retornando ao DOS
mov ah, 4Ch
int 21h
main endp
end main

```

O que acontece na memória (Exemplo)

Se o usuário digitar a palavra `OLA` (3 letras) e apertar Enter, o buffer na memória ficará assim:

Posição no Buffer	Conteúdo	Significado
buffer[0]	20	Definido por você (limite máximo)
buffer[1]	3	Preenchido pelo DOS (quantas letras o usuário digitou)
buffer[2]	'O'	Primeira letra da string
buffer[3]	'L'	Segunda letra da string
buffer[4]	'A'	Terceira letra da string
buffer[5]	13	Caractere <i>Carriage Return</i> (Enter)
buffer[6]...	'\$'	Restante dos bytes originais

Por isso, na hora de mostrar o resultado com a função `09h`, nós adicionamos `2` ao registrador `DX` (`add dx, 2`). Se apontássemos para o início do buffer, o processador tentaria ler os bytes de controle como se fossem texto, gerando caracteres estranhos na tela.